Explainable AI Project Report

Hugo Percot, Vincent Gefflaut, Titouan Le Breton

April 2026

Abstract

Mechanistic Interpretability aims to reverse-engineer neural networks by decomposing polysemantic activations into understandable features. While Sparse Autoencoders (SAEs) are the standard tool for this task, they focus on compressing static network states rather than modeling underlying computation. Sparse Transcoders (ST) mitigate this downside by reconstructing a whole part of the network (e.g. one or several layers), but it suffers from its unability to reproduce the high-rank underlying network. This report examines the *Sparse Skip Transcoder* (SST), a recently proposed evolution of STs that adds an affine skip connection to absorb baseline linear transformations. First, we benchmark three released sparse coding models on `Pythia-160m` using a compact subset of SAEBench metrics. Our results show a clear trade-off: the standard SAE achieves the strongest reconstruction performance, while the SST obtains the best sparse probing and absorption scores, with the ST typically lying in between. Furthermore, we introduce a novel structural analysis of the SST's skip connection (W_{skip}) using Singular Value Decomposition (SVD) and the Logit Lens technique. Our analysis reveals a depth-dependent functional hierarchy, from early-layer syntactic filtering to late-layer global representation pivoting. While our findings strongly justify the SST architecture, we also highlight the inherent polysemanticity of the linear skip connection and critique the limitations of the Logit Lens under representational drift. Our code is available at https://github.com/T-prog123/projet_XAI/

1 Introduction and Paper Explanation

1.1 Sparse Autoencoders vs. Transcoders

A fundamental challenge in Mechanistic Interpretability for LLMs is *polysemanticity*, where individual neurons activate for multiple, unrelated concepts [3]. Sparse Autoencoders (SAEs) address this by reconstructing activations after encoding them into a higher-dimensional, sparse latent space:

$$f(x) = W_2 \mathrm{TopK}(W_1 x + b_1) + b_2 \tag{1}$$

where

- x is the vector of activations of the layer studied
- $D = dim(x)$
- $W_1 \in R^{H \times D}$ with $H \gg D$ being the autoencoder's inner dimension
- $k \ll D$ is the sparsity parameter representing the number of active features allowed
- f is trained to reconstruct x ($f(x) \approx x$).

The "information bottleneck" created by $TopK$ is used to extract the main monosemantic features that compose the activation. However, SAEs' scope is limited, as they only learn to reconstruct intermediate activations. Thus, they cannot capture the underlying computation performed on the tokens.

Sparse Transcoders (ST) [4] offer a functional alternative. Instead of reconstructing a single activation space, they are trained to map the input of a specific network component (e.g., an MLP) directly to its output:

$$f(x) = W_2 \text{TopK}(W_1 x + b_1) + b_2 \tag{2}$$

but here x represents the *input* vector of a whole component (e.g. an MLP) of the network and f is trained to reconstruct its *output* (vector of activations).

Consequently, while SAEs capture static network states, transcoders approximate the functional circuit enacted by the component.

1.2 The Sparse Skip Transcoder (SST): Intuition and Architecture

While standard transcoders tackle a much more complex task, they suffer from capacity inefficiencies despite their functional advantages. MLPs inherently perform dense linear translations alongside non-linear computations. If a transcoder attempts to model this entire process, it wastes its limited capacity of sparse latents merely reconstructing the baseline linear map.

To resolve this, Paulo et al. [5] introduce the **Sparse Skip Transcoder (SST)**, which incorporates an affine skip connection into the architecture:

$$f(x) = W_2 \text{TopK}(W_1 x + b_1) + W_{skip} x + b_2 \tag{3}$$

Intuition: By allowing W_{skip} to absorb the network's baseline linear routing, the sparse latents (the TopK component) are freed from modeling dense background variance. This architectural prior forces the latents to exclusively capture highly *non-linear*, *sharp*, and *interpretable* features, but it relies on the strong assumption that such a decomposition correctly approximates the behavior of the MLP.

1.3 Benchmarking Methodology and Main Results

Evaluating dictionary learning models (a class to which SAEs, STs, and SSTs belong) requires balancing reconstruction fidelity against interpretability, a trade-off controlled by the number of active latents K in the sparse bottleneck. The authors benchmark SSTs against standard SAEs and STs along a Pareto frontier, utilizing both quantitative and qualitative metrics [5]:

1. **Reconstruction Fidelity:** To measure how well the sparse coder preserves model behavior, the authors patch the coder back into the original language model. They evaluate the *degradation in cross-entropy (CE) loss* over a 500K token corpus, as well as the *variance explained* by the reconstructed activations using the SAEBench framework.

2. **Automated Interpretability (Autointerp):** The interpretability of the latents is quantified using an LLM-as-a-judge pipeline [2]. An explainer model (Llama 3.1 70B) generates natural language descriptions of latent behaviors based on 40 activating examples sampled across ten quantiles. These explanations are then rigorously evaluated through *detection* and *fuzzing* tasks on unseen activating and non-activating examples, along with *simulation* scores to test predictive accuracy.

3. **Downstream Task Performance:** The authors additionally utilize SAEBench sparse probing to assess if the encoders select information relevant to classification tasks, alongside feature absorption metrics.

Main Results in the Original Paper: The SST architecture is presented as a strong alternative to standard SAEs and STs. By offloading part of the linear baseline computation to the skip connection, SSTs achieve lower cross-entropy loss degradation while also obtaining strong automated interpretability scores across several architectures. This motivates a more direct comparison on released checkpoints, which we conduct in the next section using a compact subset of SAEBench metrics.

2 Model Benchmarking

2.1 Experimental setup

Rather than retraining sparse coders from scratch or reproducing the full experimental pipeline of Paulo et al. [5], we benchmark a set of released checkpoints on a common evaluation suite. More precisely, we compare the following three pretrained models built on `Pythia-160m`, available on HuggingFace:

- `EleutherAI/Pythia-160m-SAE-k64-32k` (SAE),
- `EleutherAI/Pythia-160m-ST-k64-65k` (ST),
- `EleutherAI/Pythia-160m-SST-k64-32k` (SST).

All three models were evaluated through the SAEBench framework on a subset of metrics chosen to cover reconstruction quality, concept selection, and feature absorption. The benchmark outputs were obtained directly from the released models using fixed evaluation configurations. In particular, the core evaluation uses `OpenWebText` with context length 128, sparse probing uses the standard multi-dataset benchmark mixture, and absorption uses the first-letter absorption task.

This setup should therefore be understood as a comparative evaluation of released sparse coding models, rather than as a full reproduction of the original training and benchmarking procedure.

2.2 Selected evaluation metrics

SAEBench provides a large number of metrics, many of which are either partially redundant or too detailed for a short report. To keep the comparison compact and readable, we retain six representative metrics.

First, we consider three metrics from the core evaluation:

- **CE-loss score**, which evaluates how well model performance is preserved when the sparse coder is patched back into the model. Higher is better.
- **Explained variance**, which measures how much of the target activation variance is captured by the reconstruction. Higher is better.
- **Cosine similarity**, which measures the geometric similarity between original and reconstructed activations. Higher is better.

We then report two sparse probing metrics:

- **Sparse probing top-1 accuracy**,
- **Sparse probing top-5 accuracy**.

These indicate whether useful concepts are concentrated in a very small number of latents. Higher values indicate that the representation isolates task-relevant information more effectively.

Finally, we include one absorption metric:

- **Full absorption score**.

This measures the extent to which a general feature is absorbed into more specific ones instead of being represented cleanly. Lower is better.

Taken together, these six quantities give a compact view of the trade-off between reconstruction fidelity and interpretability-oriented behavior.

2.3 Results and interpretation

The results of the comparison are summarized in Table 1.

Model	CE-loss score ↑	Explained variance ↑	Cosine sim. ↑	Top-1 probing ↑	Top-5 probing ↑	Full absorption ↓
SAE	0.2566	0.3811	0.6260	0.6275	0.7136	0.0870
SST	-0.5211	-0.4463	-0.0192	0.6702	0.7491	0.0229
ST	-0.8184	-0.5880	-0.0739	0.6490	0.7314	0.0239

Table 1: Comparison of the three released `Pythia-160m` sparse coding models on a compact subset of SAEBench metrics.

A clear trade-off appears. On the one hand, the standard SAE is by far the strongest model on the reconstruction side: it is the only one with a positive CE-loss score, positive explained variance, and a clearly positive cosine similarity. On the other hand, the SST performs best on the two sparse probing metrics and also achieves the lowest absorption score. This suggests that, in our benchmark, the SAE preserves the original computation more faithfully, while the SST yields a cleaner concept-level organization of useful features. The ST follows the same general trend as the SST, but remains slightly behind it on the selected metrics.

Overall, these results do not show a single model dominating on every dimension. Instead, they support the idea that the skip transcoder architecture shifts part of the modeling burden away from pure reconstruction and toward a more interpretable feature organization.

3 Novel Contribution: SVD Analysis of W_{skip}

While the original paper introduced the SST architecture, they left the deeper analysis of the skip connection for future work. Taking up this suggestion, we perform Singular Value Decomposition (SVD) on the newly introduced W_{skip} matrix. We argue that this is an interesting, simple, yet inherently partial method to interpret the model's behavior via skip transcoders. Recall the SVD formulation:

$$W_{skip} = U\Sigma V^T \tag{4}$$

where U, V are unitary and Σ is diagonal. We analyzed the singular values (Σ) to determine the effective rank and "information bottleneck" of the skip connection. Furthermore, we applied the **Logit Lens** technique to the top singular vectors. By projecting the input directions (V) and output directions (U) through the model's unembedding matrix, we mapped the abstract vector spaces back to human-readable vocabulary tokens. This allows us to explicitly read what concepts are modeled by the linear transformation counterpart of the skip transcoder, giving partial insight into the role of a given layer.

3.1 Quantitative Analysis of Singular Value Decay

To evaluate the "information bottleneck" imposed by the affine skip connection, we computed the singular value spectrum (Σ) of W_{skip} across all 16 layers of the `Llama-3.2-1B` model, using the skip transcoders released by the authors. As illustrated on the left plot in Figure 1, the decay rate of the singular values reveals a distinct functional hierarchy based on network depth:

- **Early Layers (0–2):** The singular values exhibit an extremely sharp initial drop-off. This indicates that the skip connection is effectively low-rank, acting as a narrow information bottleneck that passes through only a highly specific subset of linear transformations (e.g., basic syntactic formatting) while forcing the non-linear latents to process the rest.
- **Middle Layers (5–10):** The decay curve becomes noticeably shallower, forming a tight cluster. This wider distribution of variance implies that the skip connection begins to handle a broader, more complex variety of linear conceptual shifts.

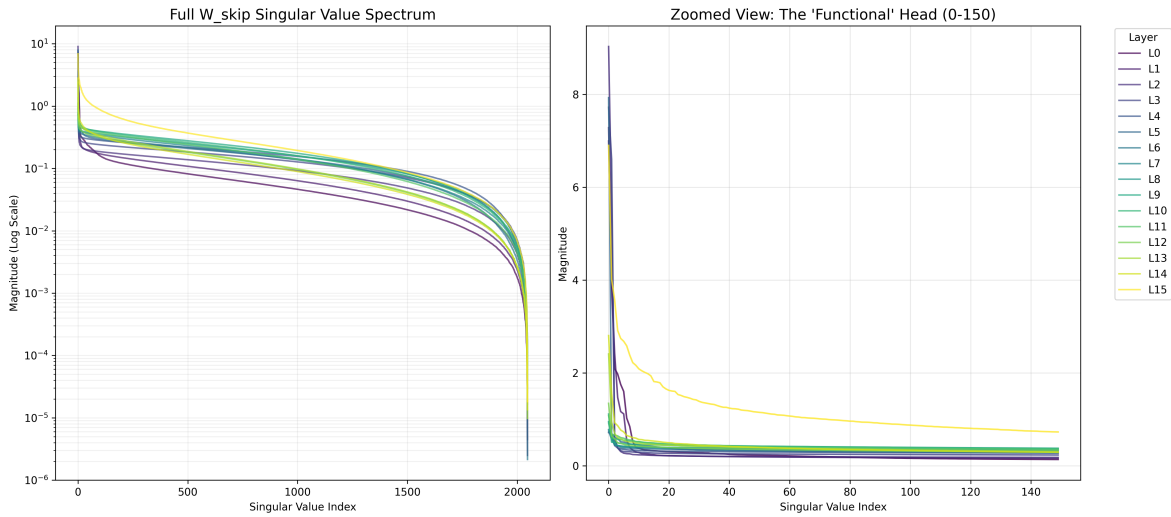

Figure 1: Magnitude of the singular values of W_{skip} for each layer of the `Llama-3.2-1B` model.

- **Late Layers (e.g., Layer 15):** The final layers exhibit a uniquely high-rank preservation, maintaining significant singular values across the majority of the dimensional space. This suggests that the final FFN blocks perform massive, distributed linear shifts to align the residual stream with the final unembedding matrix.

3.2 Qualitative Interpretation via Logit Lens

The top singular vectors extracted via SVD indicate which preferential input directions (V) are linearly transformed onto preferential output directions (U). To extract human-readable semantic mappings, we compute the similarity between these directions and the token embeddings.

Specifically, for a given singular vector:

- **Read (+):** Denotes the tokens that yield the highest positive dot product with the input direction (V). Conceptually, these are the contextual "triggers" that the skip connection is actively listening for in the residual stream.
- **Write (+):** Denotes the tokens that yield the highest positive dot product with the output direction (U). Conceptually, these represent the "action" or explicit next-token predictions that the skip connection directly injects into the residual stream, bypassing the non-linear MLP latents entirely.

Via this Logit Lens analysis, these mappings demonstrate exactly how the skip connection acts as a functional router for specific tasks. Representative examples include:

3.2.1 Layer 0: Syntactic Filtering

In the earliest layer, W_{skip} primarily handles punctuation and formatting noise.

- **Vector 1 (Read +):** `['inherited', 'idade', 'hões', 'consequence', ...]`
- **Vector 1 (Write +):** `['.', '::', ';', ':', '().']`

Interpretation: The skip connection demotes noisy, mixed-script, or inherited sub-word tokens into basic structural punctuation markers, cleaning the residual stream for subsequent non-linear processing.

3.2.2 Layer 8: Morphological Transformation

In the middle layers, the skip connection transitions to rule-based grammatical routing.

- **Vector 1 (Read +):** `['movement', 'space', 'Movement', 'movements', ...]`
- **Vector 1 (Write +):** `['ąż', 'łu', 'zeń', 'ście', 'śnie']`

Interpretation: The matrix linearly bridges abstract spatial concepts (input) to specific morphological suffixes (output), acting as an automatic syntactic translator without requiring complex non-linear logic.

3.2.3 Layer 15: Global Representation Pivoting

In the final layer, W_{skip} handles high-level representational shifts.

- **Vector 2 (Read +):** `[',', 'in', 'the', 'a', 'and', 'of', 'an']`
- **Vector 2 (Write +):** `['<Arabic>', '<Arabic>', '<Chinese>', '<Chinese>', '<Arabic>', '<Chinese>']`

Interpretation: The skip connection maps foundational English stopwords directly to entirely different linguistic scripts (e.g., Arabic, Chinese). This represents a massive, global pivot of the residual stream to prepare for multilingual text generation.

3.3 Discussion and Limitations

Our analysis provides empirical support for the Skip Transcoder (SST) architecture while also highlighting inherent limitations in both the model and our interpretability methods.

Validation of the SST Architecture: The high-rank nature of W_{skip} in later layers strongly justifies the inclusion of the affine connection. If a standard SAE or basic ST were forced to capture the massive linear shifts observed in Layer 15, it would exhaust its sparse dictionary capacity simply recreating a dense linear map. By absorbing this, W_{skip} preserves the sparse latents for genuine non-linear feature representations.

Polysemanticity of the Skip Connection: While the primary goal of transcoders is to yield monosemantic features via sparse latents, our SVD analysis proves that the W_{skip} matrix itself remains highly polysemantic. A single singular vector often handles multiple disparate tasks simultaneously (e.g., managing five different punctuation marks or multiple languages).

Limitations of the Logit Lens: Our qualitative methodology relies heavily on the Logit Lens, which projects intermediate activations directly into the final unembedding space. However, as critiqued by Belrose et al. [1], this technique suffers from "representational drift". It relies on the strong, unverified assumption that intermediate embeddings share the exact same semantic coordinate system as the final layer. In reality, the residual stream undergoes significant representational drift across depth. As observed in the Annex, many intermediate "Read" tokens manifest as multilingual fragments or sub-word noise. This indicates that middle-layer vectors do not map cleanly to the final human vocabulary without layer-specific affine translations (such as a Tuned Lens), meaning our interpretation of these intermediate mechanics may be an oversimplification or a misinterpretation of the true internal manifold.

4 Discussion and Conclusion

In this report, we critically examined the Skip Transcoder (SST), a novel dictionary learning architecture designed to isolate the functional circuits of language models. By adding an affine skip connection to a sparse transcoder, the SST effectively offloads dense linear routing, forcing its sparse latents to represent highly non-linear and monosemantic concepts.

Synthesis of Benchmarking Results: Through a comparative benchmark on released checkpoints, we evaluated the SST architecture against standard Sparse Autoencoders (SAEs) and Sparse Transcoders (STs). Our benchmarking results highlight a clear trade-off between reconstruction fidelity and interpretability-oriented behavior. The standard SAE is the strongest model on reconstruction metrics, being the only one to achieve positive CE-loss score and explained variance, and also displaying the highest cosine similarity between original and reconstructed activations. By contrast, the SST performs best on sparse probing and achieves the lowest absorption score, suggesting a cleaner organization of task-relevant concepts. The ST generally follows the same trend as the SST, but remains slightly weaker on the selected metrics. In that sense, our results do not identify a single model that dominates on every axis, but they do support the intuition that skip transcoders exchange some reconstruction strength for a better feature structure.

Structural Insights from SVD: Our novel SVD analysis provides a strong empirical and geometric justification for the authors' architectural intuition. We demonstrated that the skip connection (W_{skip}) absorbs a massive amount of linear variance, particularly in the final layers of the network where it performs high-rank global representation pivoting. If a standard sparse coder were tasked with this same transformation, it would likely exhaust its latent capacity attempting to reconstruct these dense, polysemantic shifts. However, our Logit Lens analysis also reveals that the skip connection acts as a highly polysemantic router itself, simultaneously managing multiple disparate concepts (such as formatting noise and multilingual scripts) within single singular vectors.

Limitations and Future Work: While the SST represents a significant step forward in functional interpretability, our analysis highlights critical limitations. Methodologically, our qualitative analysis assumes that intermediate embeddings share a semantic coordinate system with the final vocabulary space. The presence of sub-word noise and multilingual fragments in our intermediate Logit Lens projections strongly points to representational drift [1], suggesting that future analyses should employ layer-specific affine translators, such as the Tuned Lens, to mitigate representational mismatch. This benchmark study also has clear limitations. First, we only evaluate released checkpoints rather than retraining the models ourselves, so our conclusions remain tied to these particular pretrained instances. Second, we restrict attention to a compact subset of SAEBench metrics, which is sufficient for a meaningful comparison but does not cover the full range of evaluations considered in the literature. Finally, the compared models are not perfectly matched in dictionary size, since the ST checkpoint uses 65k latents while the SAE and SST checkpoints use 32k, which should be kept in mind when interpreting the comparison.

Ultimately, Skip Transcoders appear to provide a promising functional alternative to state-compressing SAEs, though our benchmark highlights a non-trivial trade-off between reconstruction fidelity and feature-level interpretability. Future research should investigate whether the polysemantic linear skip connection can be further factored or sparsified, and how SSTs might be utilized to map causal feature circuits across entire network architectures.

References

[1] Nora Belrose, Zach Furman, Logan Smith, Danny Halawi, Igor McKinney, Tai Muttersbach, Stella Nayak, Asher Padmakumar, David Ray, Laria Rubin, et al. Eliciting latent predictions from transformers with the tuned lens. *arXiv preprint arXiv:2303.08112*, 2023.

[2] Steven Bills, Nick Cammarata, Dan Mossing, Henk Tillman, Leo Gao, Gabriel Goh, Ilya Sutskever, Jan Leike, Jeff Wu, and William Saunders. Language models can explain neurons in language models. *OpenAI Blog*, 2023.

[3] Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermyn, Tom Conerly, Nick Turner, Cem Anil, Carson Denison, Amanda Askell, et al. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023.

[4] Jacob Dunefsky, Philippe Chlenski, and Neel Nanda. Transcoders find interpretable llm feature circuits, 2024.

[5] Gonçalo Paulo, Stepan Shabalin, and Nora Belrose. Transcoders beat sparse autoencoders for interpretability, 2025.

A Full SVD Analysis Log

The following log contains the Logit Lens projections for the top 5 singular vectors across all 16 layers of the `Llama-3.2-1B` skip transcoder. *(Note: Non-Latin characters have been replaced with descriptive tags such as `<Arabic>`, `<Chinese>`, `<Cyrillic>`, etc., to ensure proper document compilation without specialized font packages).*

```
SVD Analysis Log for meta-llama/Llama-3.2-1B
==================================================

=== LAYER 0 ===
  [Vector #1] Singular Value: 7.00
    Read (+) : ['inherited', 'idade', 'hões', 'hiba', 'Scheduled', 'éis', 'conserv', '<Cyrillic>', 'consequence', 'departed']
    Write (+): ['.', '', '.', '', '.', '::', ';', ':', '-C', '().']

  [Vector #2] Singular Value: 3.93
    Read (+) : ['<Cyrillic>', 'us', 'alo', '<Greek>', 'reason', 'embro', 'sağlamak', '347', 'ado', 'nev']
    Write (+): ['.', '.', '', ',', '.', 'du', ':', ':', '', ',']

  [Vector #3] Singular Value: 2.10
    Read (+) : [')(((', 'etc', '@yahoo', 'println', 'igh', 'py', 'lem', "',$", 'utches', '-%']
    Write (+): ['uder', 'habit', 'tablets', 'quen', 'AAP', 'abin', 'Locker', 'inda', 'DIC', '777']

  [Vector #4] Singular Value: 1.98
    Read (+) : ['endregion', 'Incontri', ';element', '.createSequentialGroup', '<Arabic>', '\xadi', '<Devanagari>', '<Chinese>', 'filetype', '<Chinese>']
    Write (+): ['erap', 'steer', 'aes', 'Moody', 'Ruby', 'mileage', 'a', 'Wiley', 'Pills', 'scape']

  [Vector #5] Singular Value: 1.75
    Read (+) : ['dana', 'uncomment', 'rtype', 'Plantae', '_hierarchy', '<Greek>', 'Mutable', '<Unknown>', '<Greek>', 'traction']
    Write (+): ['avern', '<Vietnamese>', 'ecast', 'customers', '<Cyrillic>', 'ąż', 'rome', 'feat', 'Wayback', 'asking']

=== LAYER 1 ===
  [Vector #1] Singular Value: 7.28
    Read (+) : ['SCRIBE', 'retim', 'ált', '<Cyrillic>', 'ombok', 'conto', 'olutely', 'pán', 'VILLE', '<Cyrillic>']
    Write (+): ['package', '<Chinese>', 'import', 'Tags', '(ns', '---', 'Question', 'kses', 'himself', '']

  [Vector #2] Singular Value: 3.95
    Read (+) : ['premi', 'ebay', 'Successful', '<Vietnamese>', '.ali', 'tedy', '-origin', 'leh', 'repression', '<Vietnamese>']
    Write (+): ['oten', 'urre', 'erect', 'sten', 'aji', 'oured', 'ould', 'consort', 'Dominic', 'Weiner']

  [Vector #3] Singular Value: 2.43
```

```
    Read (+) : ['aldi', 'acker', 'omin', 'ustr', 'uner', 'eros', 'essen', 'agn', '<Cyrillic>', 'heim']
    Write (+): ['ilk', 'noch', 'aturing', 'urls', 'losing', 'itored', 'herited', 'dra', 'plated', 'mods']

  [Vector #4] Singular Value: 1.47
    Read (+) : ['<Cyrillic>', '<Korean>', 'ads', 'achers', 'ang', 'erts', 'conj', 'again', 'ems', 'again']
    Write (+): ['<Vietnamese>', 'encion', 'imson', 'isser', '<Cyrillic>', 'holm', 'imen', 'rlen', 'ukan', 'ffen']

  [Vector #5] Singular Value: 1.17
    Read (+) : ['olf', 'Tai', 'made', 'Tai', 'blo', 'copied', 'Making', 'Terr', 'iske', 'Tribute']
    Write (+): ['klein', 'illos', 'Teknik', 'astically', 'lä', '<Unknown>', 'indic', 'estr', 'ework', '<Vietnamese>']

=== LAYER 2 ===
  [Vector #1] Singular Value: 9.03
    Read (+) : ['ountains', 'fixed', 'Tex', 'fix', '(force', 'Stay', 'overlook', '_USE', 'Sie', '_cur']
    Write (+): ['niż', 'diagnosed', 'PMC', 'Macedonia', 'diagnosis', 'ful', '<Chinese>', 'familiar', 'osoph', '<Japanese>']

  [Vector #2] Singular Value: 5.54
    Read (+) : ['.Bundle', 'YK', 'ød', 'år', 'mute', 'úi', '<Cyrillic>', 'elps', 'URT', 'asjon']
    Write (+): ['<![', '.linkedin', '<Japanese>', 'itself', '_override', 'aising', 'amen', 'elen', 'Amen', 'force']

  [Vector #3] Singular Value: 0.57
    Read (+) : ['ourselves', 'Percent', 'estr', 'unseren', '<Chinese>', 'Datum', 'efd', '!_', '-trigger', 'triggers']
    Write (+): ['ľ', 'eled', '\xadi', 'ąż', 'ół', 'ęp', 'że', 'śnie', 'asto', '<Cyrillic>']

  [Vector #4] Singular Value: 0.38
    Read (+) : ['ält', ':\\"', '.tencent', 'ån', ':"', 'oje', '\\"', '-Speed', ',\\"', '..."']
    Write (+): ['Reds', 'artin', 'usic', 'icine', 'ouns', 'ByPrimaryKey', 'oogle', 'eriod', 'arrays', 'spell']

  [Vector #5] Singular Value: 0.31
    Read (+) : ['age', 'const', 'ance', 'soil', ']:', 'idents', 'Forrest', 'bacteria', '<Chinese>', 'edelta']
    Write (+): ['<Cyrillic>', '+",', '<Cyrillic>', '<Cyrillic>', '<Chinese>', 'traps', 'ská', '<Chinese>', 'trap', '<Cyrillic>']

=== LAYER 3 ===
  [Vector #1] Singular Value: 7.72
    Read (+) : ['Fen', '_IMPLEMENT', '<Vietnamese>', 'visor', '.gl', '_MOD', 'fulness', 'ose', 'usb', 'llib']
    Write (+): ['interpreted', 'means', 'subdivisions', 'Jama', 'means', 'heating', 'Durant', 'arenas', 'energy', 'penetration']

  [Vector #2] Singular Value: 6.60
    Read (+) : ['że', 'ół', 'ście', 'eled', 'ýš', 'ľ', 'ąż', 'ěž', 'rowad', 'eczy']
    Write (+): ['vetica', 'ctl', 'supporters', '!=', 'Bulldogs', 'Supporters', 'endorsements', 'Redirect', '<Japanese>', '<Cyrillic>']
```

```
  [Vector #3] Singular Value: 0.59
    Read (+) : ['<Chinese>', 'onet', 'hecy', 'oner', 'charged', 'discharged', 'Called', 'bills', 'RF', 'trade']
    Write (+): ['oundation', '/cc', '<Thai>', 'oq', '_ste', 'finder', '<S', 'fried', '(force', 'sk']

  [Vector #4] Singular Value: 0.39
    Read (+) : ['\\\\/', 'geological', '<Chinese>', '\\\\\\', '[...]', 'verity', '$("<', '\\\\', 'mez', 'infertility']
    Write (+): ['Priv', 'edor', 'Priv', 'keyed', 'centres', 'failures', 'Centers', 'fix', 'Hij', 'tied']

  [Vector #5] Singular Value: 0.32
    Read (+) : ['edelta', 'isseur', 'avg', 'merc', 'fast', 'osaic', 'statement', 'chair', '<Korean>', 'Cha']
    Write (+): ['lač', 'ikler', '.Bundle', '<Devanagari>', 'INED', 'onden', 'oloj', 'dde', 'ła', 'ije']

=== LAYER 4 ===
  [Vector #1] Singular Value: 0.71
    Read (+) : ['ausp', '<Japanese>', '<Cyrillic>', '<Korean>', 'signals', 'presup', '.ul', 'wig', 'heck', '<Arabic>']
    Write (+): ['<Cyrillic>', 'yna', 'ście', 'że', 'ľ', 'subclass', 'asto', 'pty', 'ź', 'orex']

  [Vector #2] Singular Value: 0.68
    Read (+) : ['elps', 'imei', 'acos', 'Pant', '<Cyrillic>', '<Arabic>', 'engel', 'getLocale', 'eng', 'protect']
    Write (+): ['_IMPLEMENT', 'igham', 'istics', 'exc', 'ricular', 'buf', 'ibern', 'complexity', '<Chinese>', '.mousePosition']

  [Vector #3] Singular Value: 0.43
    Read (+) : ['<Thai>', 'coni', 'ść', 'HOLDER', '<Arabic>', 'ials', 'usc', 'cellular', 'eczy', 'standing']
    Write (+): ['insensitive', 'ublish', 'exporting', 'answers', 'zoom', '-IS', 'dsa', 'matters', 'invoking', 'unte']

  [Vector #4] Singular Value: 0.38
    Read (+) : ['<Cyrillic>', 'nom', 'amour', '-normal', '.newLine', 'niejs', '<Cyrillic>', 'vě', 'Spicer', 'atto']
    Write (+): ['circum', 'prom', 'USB', 'dyn', 'summoned', 'Saving', 'Stuff', 'Hero', 'rumors', 'uem']

  [Vector #5] Singular Value: 0.36
    Read (+) : ['bach', 'ighbours', 'Err', '_EVAL', 'apan', 'iedo', 'ERR', 'Ras', 'Forward', 'convo']
    Write (+): ['éné', '_animation', 'genders', 'itioner', 'sexes', 'cid', 'ghi', '<Arabic>', 'age', '="#']

=== LAYER 5 ===
  [Vector #1] Singular Value: 7.93
    Read (+) : ['mute', 'elps', '.Bundle', '\\Controllers', 'imei', 'YK', '.amazon', 'å', 'IELD', '<Vietnamese>']
    Write (+): ['tall', 'Nam', 'cond', 'NaN', 'Radio', 'Shop', 'pane', 'miss', 'mes', '.chunk']

  [Vector #2] Singular Value: 0.70
    Read (+) : ['ISTER', 'FOOT', 'plevel', 'plate', 'ificio', 'LTD', 'Solution', 'UMMY', '.assertIn', 'ambassadors']
    Write (+): ['å', '<Symbol>', 'ar', '_ste', '\\Controllers', 'Jack', '(exports', 'mute', '(force', 'fried']
```

```
  [Vector #3] Singular Value: 0.60
    Read (+) : ['peri', '<Korean>', 'yth', '(Print', 'roz', 'apor', 'elin', 'offence', 'QRST', 'amil']
    Write (+): ['rypto', 'exhibitions', 'aggio', 'gl', '<Japanese>', 'iamo', 'ICAST', 'iest', 'IENT', '<Vietnamese>']

  [Vector #4] Singular Value: 0.59
    Read (+) : ['<Greek>', 'ardless', 'cray', 'gency', 'ubby', 'rossover', 'inish', 'naï', '.swing', 'Andrews']
    Write (+): ['<Chinese>', 'ém', 'localized', 'elong', 'lash', 'fame', 'machine', 'rem', 'Malk', 'DLC']

  [Vector #5] Singular Value: 0.51
    Read (+) : ['ardy', 'itus', 'orm', 'Import', 'Ris', 'inherit', 'import', '<Cyrillic>', 'ecs', '<Chinese>']
    Write (+): ['<Chinese>', 'nar', 'regist', '<Japanese>', 'horizontal', '<Vietnamese>', '<Korean>', 'mini', 'IDGET', 'Spaces']

=== LAYER 6 ===
  [Vector #1] Singular Value: 0.74
    Read (+) : ['.webkit', 'exual', 'link', 'ashes', 'faction', 'rog', '.telegram', 'gn', 'tud', 'lander']
    Write (+): ['(force', '\\Controllers', 'Rai', 'force', 'å', 'acos', '<Vietnamese>', 'utations', 'imei', '(exports']

  [Vector #2] Singular Value: 0.57
    Read (+) : ['dio', 'vey', '<Chinese>', '_den', 'Fin', 'Cri', 'den', 'opard', 'bin', 'vice']
    Write (+): ['diff', 'Wi', 'quotient', 'vog', 'McCorm', ':x', 'usb', 'Dal', 'adm', 'Addison']

  [Vector #3] Singular Value: 0.44
    Read (+) : ['Tenn', 'Killer', 'acci', 'greatness', 'annie', 'Pricing', 'Vance', 'umeric', 'uve', '.bind']
    Write (+): ['hu', 'scop', 'isan', 'achine', 'ritch', 'Sound', '<Chinese>', 'BU', '_sound', 'obus']

  [Vector #4] Singular Value: 0.44
    Read (+) : ['jury', 'shalt', 'Coffee', 'uler', '.ly', 'Cast', 'ucu', 'Injury', '.dtd', '<Cyrillic>']
    Write (+): ['prostřed', 'interv', 'xico', 'posit', 'ifer', 'icus', 'ior', 'flag', '<Chinese>', 'uv']

  [Vector #5] Singular Value: 0.39
    Read (+) : ['ğı', 'già', 'kem', 'Asked', 'umo', 'kami', 'Asked', 'invert', 'isl', '/extensions']
    Write (+): ['UPC', 'capt', 'flakes', 'replen', 'oting', '<Chinese>', 'ainted', 'bothers', 'hs', 'otec']

=== LAYER 7 ===
  [Vector #1] Singular Value: 0.78
    Read (+) : ['achts', '<Chinese>', 'aç', 'å', 'unstoppable', 'Falls', 'äche', 'UNKNOWN', '.Office', 'vůbec']
    Write (+): ['fulness', '<Cyrillic>', '<Chinese>', 'ream', 'phin', 'ose', '://%', 'Fen', 'Gn', '_IMPLEMENT']

  [Vector #2] Singular Value: 0.63
    Read (+) : ['chine', 'Sid', '-rights', '<Unknown>', '-IS', '<Cyrillic>', 'elps', 'ièrement', 'toast', '("-']
```

```
    Write (+): ['ście', 'ół', 'ěl', 'eled', 'że', 'ży', 'isko', 'ěž', 'niejs', 'ť']

  [Vector #3] Singular Value: 0.45
    Read (+) : ['Governance', 'čná', 'ingle', '<Cyrillic>', 'Qualified', 'lied', '<Cyrillic>', '<Chinese>', 'ampp', 'acyj']
    Write (+): ['usp', 'isce', 'Springer', 'chapter', 'ebi', 'Rev', 'fa', 'Hubb', 'base', 'mud']

  [Vector #4] Singular Value: 0.43
    Read (+) : ['<Arabic>', 'caution', 'Lights', 'lide', 'floors', 'sched', 'lots', 'Lights', 'bart', 'floor']
    Write (+): ['usta', 'uc', 'ailable', 'rita', 'ugas', 'ritz', 'aguay', 'ult', 'promin', '<Chinese>']

  [Vector #5] Singular Value: 0.42
    Read (+) : ['isku', 'üle', 'Polic', '.Clear', '-clear', '<Japanese>', 'ulp', 'earer', 'chair', 'Setter']
    Write (+): ['lew', 'spray', 'storms', 'ronym', 'Walter', 'scri', 'scrub', 'multin', 'capability', 'ognito']

=== LAYER 8 ===
  [Vector #1] Singular Value: 0.95
    Read (+) : ['.Static', 'Mov', 'movement', '13', 'space', 'Movement', '2', ',', 'van', 'movements']
    Write (+): ['ąż', 'łu', 'zeń', 'ście', 'ąd', '\xadi', 'Ľ', 'ół', 'śnie', 'ń']

  [Vector #2] Singular Value: 0.69
    Read (+) : ['orex', 'egl', 'uci', 'ilst', 'flake', 'isti', '-UA', 'yna', 'IGN', 'odes']
    Write (+): ['.Bundle', 'mute', 'elps', 'å', '\Controllers', 'kou', '(force', 'getLocale', 'nye', 'anse']

  [Vector #3] Singular Value: 0.48
    Read (+) : ['Flint', 'discontin', 'Lee', 'Den', 'fan', 'Mis', 'Touches', 'Leh', 'flavour', 'clear']
    Write (+): ['checked', 'dum', 'asia', 'Las', 'ults', 'check', 'ulas', 'sac', 'unta', 'sou']

  [Vector #4] Singular Value: 0.44
    Read (+) : ['Programmer', 'ae', 'Goals', 'ilha', 'iane', 'oram', 'orz', 'iaz', 'aniu', '<Unknown>']
    Write (+): ['ash', 'yst', '<Arabic>', 'Buildings', 'okes', '{},', 'SelectionMode', 'destroyer', '.cfg', '<Chinese>']

  [Vector #5] Singular Value: 0.43
    Read (+) : ['<Cyrillic>', '<Chinese>', 'fir', 'isecond', 'PWD', 'simp', 'ocene', 'IW', '<Korean>', 'Forums']
    Write (+): ['dims', 'olt', '<Cyrillic>', '<Unknown>', 'terra', 'dims', '<Korean>', 'pt', 'orges', 'igmat']

=== LAYER 9 ===
  [Vector #1] Singular Value: 1.11
    Read (+) : ['.projects', 'airs', '.setter', 'adol', 'yb', 'Royals', '])[', 'GT', 'onymous', 'ariat']
    Write (+): ['ży', 'ła', '\xadi', 'ije', 'če', '.Bundle', 'asto', '<Arabic>', 'ście', 'niejs']

  [Vector #2] Singular Value: 0.51
```

```
    Read (+) : ['<Devanagari>', 'lak', 'fl', '<Chinese>', 'pled', 'etype', 'znám', 'IDES', 'Sense', 'ESCO']
    Write (+): ['olon', '<Unknown>', 'Wa', 'zew', '<Chinese>', '.toolbox', 'Nab', 'relev', 'Prescott', 'Mission']

  [Vector #3] Singular Value: 0.51
    Read (+) : ['issions', 'ichten', 'ission', 'reur', '<Chinese>', 'tings', '<Chinese>', '<Unknown>', '<Korean>', 'pline']
    Write (+): ['eh', 'anza', 'iez', 'rho', 'introduction', 'Sie', 'Collection', 'ieme', 'eu', 'eyh']

  [Vector #4] Singular Value: 0.49
    Read (+) : ['constitution', 'ANN', 'assic', '<Arabic>', 'erior', 'Associations', 'Tips', '-responsive', 'PAT', 'SCR']
    Write (+): ['soft', 'Black', 'black', 'Kraft', 'draft', 'de', '<Cyrillic>', 'Earth', 'punct', 'runApp']

  [Vector #5] Singular Value: 0.48
    Read (+) : ['Overnight', 'overnight', 'Freund', 'zet', 'orgen', 'acos', 'Hal', 'Observ', 'Advertisement', 'Glow']
    Write (+): ['<Greek>', '<Chinese>', 'insk', 'hoff', 'ins', 'elper', 'limiting', 'ackers', 'Second', '<Greek>']

=== LAYER 10 ===
  [Vector #1] Singular Value: 0.92
    Read (+) : ['RAP', 'sen', 'asma', 'onte', '<Korean>', '<Vietnamese>', '/bar', 'pursued', '<Chinese>', 'ADV']
    Write (+): ['delta', 'gl', 'innamon', '/etc', 'dw', 'estad', 'rumors', 'diff', 'gan', 'Beam']

  [Vector #2] Singular Value: 0.59
    Read (+) : ['express', 'ome', 'propri', 'alle', 'EXPRESS', 'Express', 'Cardinal', 'Trade', 'TOR', 'Appro']
    Write (+): ['repr', 'elig', 'uest', 'agers', 'brook', 'orate', '<Chinese>', 'order', 'otype', 'window']

  [Vector #3] Singular Value: 0.58
    Read (+) : ['COVID', 'Covid', 'Zoom', 'WL', 'holds', 'mn', 'ulos', 'bars', 'zoom', 'COVID']
    Write (+): ['itra', 'ITA', 'ąd', 'çe', 'śmy', 'uell', 'rosis', '.program', 'ues', 'idas']

  [Vector #4] Singular Value: 0.57
    Read (+) : ['shooter', 'Carrier', 'Legend', 'Eugene', 'Shooter', 'Gam', 'lane', 'Hor', 'xx', '.contents']
    Write (+): ['/login', 'IfNeeded', '_windows', 'Juliet', 'cylinders', 'deser', 'urg', '_SELECTION', 'ataka', '.windows']

  [Vector #5] Singular Value: 0.56
    Read (+) : ['Signed', 'HS', '_IA', 'IPA', 'Signs', 'vido', 'LS', 'anos', 'Vue', 'Startup']
    Write (+): ['verz', '<Unknown>', 'gest', '<Cyrillic>', '<Unknown>', '<Japanese>', '<Cyrillic>', 'Monterey', '<Arabic>', '_filenames']

=== LAYER 11 ===
  [Vector #1] Singular Value: 1.08
    Read (+) : ['Dumpster', 'anz', 'Prison', '<Cyrillic>', '<Cyrillic>', '<Cyrillic>', '655', 'Gest', "'aff", '/help']
    Write (+): ['innamon', 'Shea', '<Unknown>', '<Unknown>', '<Cyrillic>', '&utm', '<Chinese>', 'Huffman', '<Chinese>', '<Chinese>']
```

```
  [Vector #2] Singular Value: 0.68
    Read (+) : ['ubbles', 'ught', 'ughty', 'rought', '<Cyrillic>', '.inst', 'amarin', 'ejected', 'Thai', 'pa']
    Write (+): ['indis', 'fled', '<Chinese>', 'shrine', 'reass', 'Mango', '<Unknown>', '<Cyrillic>', 'tiv', 'lv']

  [Vector #3] Singular Value: 0.66
    Read (+) : ['elman', 'elled', 'endcode', 'refunded', 'mdl', 'DPS', 'rejoice', 'vej', 'entitled', 'ernals']
    Write (+): ['@(', '@(', 'mek', 'Fiesta', 'desta', 'cake', 'endimento', 'lige', 'soaking', "(\\'"]

  [Vector #4] Singular Value: 0.64
    Read (+) : ['ek', '-half', 'uch', 'ovan', 'ank', '<Chinese>', '-equ', 'ui', '-wh', 'unk']
    Write (+): ['alion', 'azioni', '185', 'elijke', 'iane', 'ouble', 'inth', '431', '<Cyrillic>', 'erah']

  [Vector #5] Singular Value: 0.61
    Read (+) : ['qua', '<Vietnamese>', 'itra', '=end', 'itness', 'orney', 'hlen', 'Nacht', 'vocals', 'ade']
    Write (+): ['<Cyrillic>', '<Devanagari>', "B", 'uD', 'posto', '.xhtml', 'ssql', 'Regs', '<Chinese>', '<Unknown>']

=== LAYER 12 ===
  [Vector #1] Singular Value: 1.34
    Read (+) : ['anz', '<Cyrillic>', 'icker', '<Cyrillic>', '<Cyrillic>', '<Cyrillic>', 'lear', 'Severity', 'Insensitive', '-router']
    Write (+): ['innamon', '.arc', 'Aud', 'Shea', '.glide', '<Chinese>', '&utm', '<Unknown>', '/etc', '<Unknown>']

  [Vector #2] Singular Value: 0.64
    Read (+) : ['McMaster', '&q', '<Vietnamese>', 'dio', 'Mayo', 'sed', '.imgur', '.mit', 'appen', '.easy']
    Write (+): ['heartbeat', 'venge', 'unrecognized', 'xico', 'balk', 'Scotland', 'Visibility', 'causa', 'volution', 'UNKNOWN']

  [Vector #3] Singular Value: 0.58
    Read (+) : ['aled', 'eled', 'iel', 'Tak', 'Tes', 'nick', 'er', 'ayla', 'ellt', 'nick']
    Write (+): ['Accessory', 'eer', 'hips', 'fixture', 'kits', 'sch', 'Bols', 'ein', 'ddit', '<Vietnamese>']

  [Vector #4] Singular Value: 0.49
    Read (+) : ['rica', 'nesia', 'Stateless', '/channel', 'rique', '<Arabic>', 'Visibility', '<Korean>', 'bies', 'cakes']
    Write (+): ['<Arabic>', 'etti', 'ickers', 'urt', 'Elon', '?page', 'iques', '<Chinese>', 'Serena', 'rub']

  [Vector #5] Singular Value: 0.48
    Read (+) : ['en', 'PEND', 'elle', '-or', 'e', '-p', 'clo', 'ple', '-h', 'a']
    Write (+): ['zburg', '<Arabic>', 'tls', 'icity', 'oine', 'urf', 'ussels', 'sburg', 'ein', 'POP']

=== LAYER 13 ===
  [Vector #1] Singular Value: 2.41
    Read (+) : ['Bowie', 'D', 'été', 'ète', '<Chinese>', '_channel', 'použití', 'Js', '-channel', 'Mil']
    Write (+): ['łem', 'ält', 'zą', 'ła', 'ży', 'ły', 'zeń', 'adamente', 'ści', '\xadi']
```

```
  [Vector #2] Singular Value: 0.94
    Read (+) : ['Kraj', 'olit', 'Wayback', 'Nagar', 'Yad', 'alue', 'Tencent', 'ambil', 'vil', 'hexadecimal']
    Write (+): [',', 'if', 'y', '', 'too', 'same', '.', '(', 'while', ':']

  [Vector #3] Singular Value: 0.70
    Read (+) : ['guest', 'pointer', 'Stateless', 'Cristiano', 'uzzi', 'Anadolu', 'spell', 'guest', 'Donne', 'McCabe']
    Write (+): ['Ack', 'Krist', '<Vietnamese>', '<Thai>', '<Chinese>', 'Florian', '<Korean>', '<Unknown>', 'addle', 'etter']

  [Vector #4] Singular Value: 0.60
    Read (+) : ['olest', 'tsy', 'tok', '<Cyrillic>', 'Dost', 'lust', 'Reco', 'Vect', 'deaux', 'ostel']
    Write (+): ['', '.', '.', '<|begin_of_text|>', '', '', ',', '...', 'that', 'for']

  [Vector #5] Singular Value: 0.58
    Read (+) : ['louder', 'emean', 'cstdlib', 'kidd', '<Cyrillic>', 'vocational', 'unicip', 'sounding', 'lion', 'Vladim']
    Write (+): ['', 'Andrew', 'John', '', '.', 'Hof', 'American', '.', ';', 'Jill']

=== LAYER 14 ===
  [Vector #1] Singular Value: 2.80
    Read (+) : ['sns', 'characterized', '{\\"', '<Devanagari>', 'ê', 'hä', 'unya', 'ML', 'ança', 'Ã']
    Write (+): ['łem', 'adamente', 'ält', 'ży', 'ści', 'ności', 'že', 'się', 'ši', 'šem']

  [Vector #2] Singular Value: 1.56
    Read (+) : ['ture', 'olit', 'halt', 'üzel', 'Sabha', 'lluminate', 'YRO', 'OMPI', 'orate', 'RITE']
    Write (+): ['', 'in', 'a', '(', 'the', 'I', ',', 'A', 'N', 'and']

  [Vector #3] Singular Value: 0.93
    Read (+) : ['ful', 'tor', 'overs', 'ow', '<Cyrillic>', 'ak', 'fully', 'utr', 'nes', '-s']
    Write (+): ['lessness', 'pper', 'beck', 'pack', 'ago', 'illac', '<Thai>', '<Cyrillic>', 'ful', 'AGO']

  [Vector #4] Singular Value: 0.86
    Read (+) : ['iked', 'verb', 'ū', 'loh', 'ig', 'emin', 'OLT', 'arn', '313', 'lok']
    Write (+): ['comp', 'cryst', 'corner', 'successful', '<Korean>', 'idores', 'snow', 'reasonable', 'preview', 'fraction']

  [Vector #5] Singular Value: 0.77
    Read (+) : ['Gallup', '<Chinese>', 'Philips', '254', 'uro', 'Jenkins', 'Rück', 'freely', '<Unknown>', 'leaf']
    Write (+): ['ilos', 'Minds', '<Japanese>', 'cept', '<Chinese>', 'Examiner', 'rama', 'Distrib', '<Chinese>', '<Arabic>']

=== LAYER 15 ===
  [Vector #1] Singular Value: 6.90
    Read (+) : ['Annunci', 'juana', 'ribbon', 'pornos', 'prostitut', 'parator', 'PS', 'ubat', 'Čer', 'OMAP']
```

```
  Write (+): ['<|begin_of_text|>', '', '', '', '', '', '<Arabic>', '', '', '']

[Vector #2] Singular Value: 4.02
  Read (+) : [',', 'in', 'the', 'a', 'and', '', 'of', '(', 'de', 'an']
  Write (+): ['<Devanagari>', '<Arabic>', '<Chinese>', '<Chinese>', '<Arabic>', '<Chinese>', '<Chinese>', '<Arabic>', '<Thai>', '<Devanagari>']

[Vector #3] Singular Value: 3.57
  Read (+) : ['())', 'specs', 'StringValue', 'Radians', '<Cyrillic>', 'imentary', 'mmo', 'imli', '<Cyrillic>', 'ulares']
  Write (+): ['Se', 'El', 'Em', 'the', 'En', 'Ch', 'Tr', 'In', 'Har', 'Ad']

[Vector #4] Singular Value: 2.91
  Read (+) : ['', '(', ',', '', '.', '/', '(', '[', ':', '-']
  Write (+): ['?', ').</', ':<?', '‘.‘', ',{"', '?<', '*</', '"\';', "'||", '."</']

[Vector #5] Singular Value: 2.74
  Read (+) : ['\u200b\u200b', '\u200c', '\ufeff', "\\'", '\u200b', "\\'", ' ', 'WAN', 'Alv', '&apos']
  Write (+): ['OUNDS', 'AGING', '<Chinese>', '<Unknown>', 'Propagation', 'OUNTER', 'RING', 'USES', '<Cyrillic>', '<Arabic>']
```